

STOCK MARKET ANALYSIS PROJECT REPORT

Name: Gunjan Singh

Course: Data Analyst

Tools Used: MySQL, Tableau

Introduction

The stock market generates a large amount of financial data every day. Analysing this data helps investors and analysts understand market trends and make informed decisions.

This project focuses on analysing historical stock price data using *MySQL for data analysis* and *Tableau for data visualization*. The goal is to identify trends, calculate key metrics, and present insights through an interactive dashboard.

Problem Statement

The objective of this project is to analyse historical stock price data and identify trends in stock performance. Using SQL queries, important metrics such as average price, daily return, and trading volume are calculated. Tableau is then used to visualize these results and create an interactive dashboard for better understanding of the stock market behaviour.

Dataset Description

The dataset used in this project contains historical stock market data. The dataset includes the following columns:

- Ticker: Stock symbol of the company

- Date: Trading date
- Open: Opening price of the stock
- High: Highest price of the stock during the day
- Low: Lowest price during the trading session
- Close: Closing price of the stock
- Adj Close: Adjusted closing price
- Volume: Number of shares traded

This dataset helps analyse stock price movements and trading activity.

Tools Used

The following tools were used in this project:

MySQL-

Used for storing the dataset and performing SQL queries for data analysis.

Tableau-

Used for creating visualizations and interactive dashboards to represent stock trends.

Data Analysis using SQL

SQL was used to analyse the stock dataset and extract meaningful insights.

Some of the important queries performed include:

1. Highest Closing Price

```
SELECT MAX (close_price)
FROM stock_data;
```

2. Average Closing Price

```
SELECT AVG (close_price)
FROM stock_data;
```

3. Daily Return

```
SELECT stock_date, close_price - open_price AS daily_return
FROM stock_data;
```

4. Percentage Return

```
SELECT stock_date,  
  
       ((close_price - open_price) / open_price) * 100 AS  
percentage_return  
  
FROM stock_data;
```

5. Highest Trading Volume

```
SELECT stock_date, volume  
  
FROM stock_data  
  
ORDER BY volume DESC  
  
LIMIT 1;
```

```
1 • CREATE TABLE stocks (  
2     stock_date DATE,  
3     company VARCHAR(50),  
4     open_price FLOAT,  
5     high_price FLOAT,  
6     low_price FLOAT,  
7     close_price FLOAT,  
8     volume BIGINT  
9 );  
10 • SHOW TABLES;  
11 • CREATE TABLE stock_market.stocks_test (  
12     id INT  
13 );  
14 • CREATE TABLE stock_market.stock_data (  
15     stock_date DATE,  
16     company VARCHAR(50),  
17     open_price FLOAT,  
18     high_price FLOAT,  
19     low_price FLOAT,  
20     close_price FLOAT,  
21     volume BIGINT  
22 );  
23 • SHOW TABLES;  
24 • DESCRIBE stock_data;  
25 • SET GLOBAL local_infile = 1;  
26 • LOAD DATA LOCAL INFILE 'C:\\Users\\asus\\Downloads\\stocks.csv'  
27 INTO TABLE stock_data  
28 FIELDS TERMINATED BY ','  
29 ENCLOSED BY ''''  
  
29 ENCLOSED BY ''''  
30 LINES TERMINATED BY '\\n'  
31 IGNORE 1 ROWS  
32 (stock_date, company, open_price, high_price, low_price, close_price, volume);
```

```

1 • USE stock_market;
2 • INSERT INTO stock_data
  (company, stock_date, open_price, high_price, low_price, close_price, volume)
3 VALUES
4 ('AAPL', STR_TO_DATE('14-02-2023', '%d-%m-%Y'), 152.12, 153.77, 150.85, 152.59, 61707600),
5 ('AAPL', STR_TO_DATE('15-02-2023', '%d-%m-%Y'), 153.11, 155.50, 152.88, 155.33, 65573800),
6 ('AAPL', STR_TO_DATE('16-02-2023', '%d-%m-%Y'), 154.89, 156.30, 153.45, 153.71, 68167900),
7 ('AAPL', STR_TO_DATE('17-02-2023', '%d-%m-%Y'), 152.50, 153.99, 151.32, 152.55, 59144100),
8 ('AAPL', STR_TO_DATE('20-02-2023', '%d-%m-%Y'), 150.22, 151.88, 149.67, 150.88, 53388400),
9 ('AAPL', STR_TO_DATE('21-02-2023', '%d-%m-%Y'), 149.90, 150.99, 148.12, 148.47, 58867200),
10 ('AAPL', STR_TO_DATE('22-02-2023', '%d-%m-%Y'), 148.77, 149.88, 147.55, 148.91, 51011300),
11 ('AAPL', STR_TO_DATE('23-02-2023', '%d-%m-%Y'), 149.10, 150.66, 148.44, 149.40, 47612200),
12 ('AAPL', STR_TO_DATE('24-02-2023', '%d-%m-%Y'), 150.01, 151.45, 149.90, 151.03, 55438800),
13 ('AAPL', STR_TO_DATE('27-02-2023', '%d-%m-%Y'), 151.20, 153.00, 150.75, 152.88, 60277400),
14
15
16 ('AAPL', STR_TO_DATE('28-02-2023', '%d-%m-%Y'), 153.45, 154.10, 151.80, 152.12, 64882000),
17 ('AAPL', STR_TO_DATE('01-03-2023', '%d-%m-%Y'), 151.99, 153.66, 151.44, 153.21, 59022000),
18 ('AAPL', STR_TO_DATE('02-03-2023', '%d-%m-%Y'), 153.77, 155.30, 153.10, 154.98, 62001100),
19 ('AAPL', STR_TO_DATE('03-03-2023', '%d-%m-%Y'), 155.20, 156.44, 154.60, 156.31, 63387000),
20 ('AAPL', STR_TO_DATE('06-03-2023', '%d-%m-%Y'), 156.10, 157.88, 155.42, 157.02, 57944100),
21 ('AAPL', STR_TO_DATE('07-03-2023', '%d-%m-%Y'), 157.33, 158.20, 156.11, 156.89, 54877000),
22 ('AAPL', STR_TO_DATE('08-03-2023', '%d-%m-%Y'), 156.44, 157.99, 155.77, 157.50, 51100300),
23 ('AAPL', STR_TO_DATE('09-03-2023', '%d-%m-%Y'), 157.88, 159.10, 156.66, 158.88, 59388400),
24 ('AAPL', STR_TO_DATE('10-03-2023', '%d-%m-%Y'), 159.20, 160.50, 158.40, 159.77, 62044700),
25 ('AAPL', STR_TO_DATE('13-03-2023', '%d-%m-%Y'), 160.10, 161.88, 159.00, 161.02, 64277500),
26
27 ('AAPL', STR_TO_DATE('14-03-2023', '%d-%m-%Y'), 161.50, 162.30, 160.88, 161.90, 57788000),
28 ('AAPL', STR_TO_DATE('15-03-2023', '%d-%m-%Y'), 162.20, 163.99, 161.44, 163.77, 68900300),
29 ('AAPL', STR_TO_DATE('16-03-2023', '%d-%m-%Y'), 163.88, 165.12, 162.99, 164.55, 71220000),

```

- SELECT *
FROM stock_data
ORDER BY stock_date;
- SELECT
stock_date,
ROUND(((close_price - open_price) / open_price) * 100, 2) AS percentage_return
FROM stock_data
ORDER BY stock_date;
- SELECT
stock_date,
close_price,
ROUND(
AVG(close_price) OVER (
ORDER BY stock_date
ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
) , 2
) AS moving_avg_7_days
FROM stock_data;
- SELECT stock_date, close_price
FROM stock_data
ORDER BY close_price DESC
LIMIT 1;
- SELECT stock_date, close_price
FROM stock_data
ORDER BY close_price ASC
LIMIT 1;
- SELECT SUM(volume) AS total_volume
FROM stock_data;

```

SELECT
    MIN(stock_date) AS start_date,
    MAX(stock_date) AS end_date,
    MIN(close_price) AS min_price,
    MAX(close_price) AS max_price
FROM stock_data;

SELECT stock_date, volume
FROM stock_data
ORDER BY volume DESC
LIMIT 1;

SELECT COUNT(*) FROM stock_data;

```

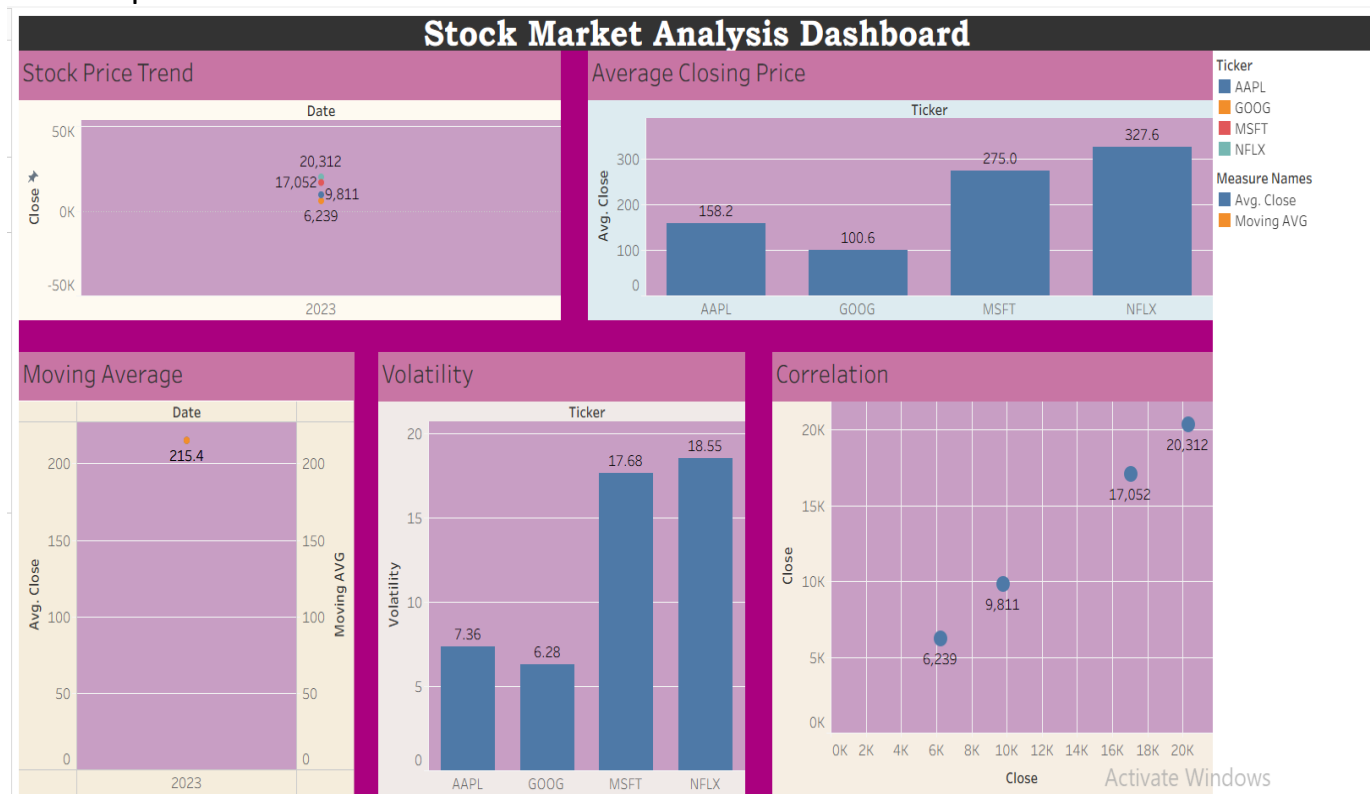
Data Visualization using Tableau

After analysing the dataset using SQL, Tableau was used to create visualizations and an interactive dashboard.

The following visualizations were created:

- Stock Price Trend Line Chart
- Average Closing Price Bar Chart
- Moving Average Line Chart
- Trading Volume Analysis
- Correlation Scatter Plot

These charts help in understanding stock performance and identifying patterns in stock price movement.



Key Insights

- a) The stock showed an overall upward trend during the analysed period.
- b) The highest closing price was observed during the later part of the dataset, indicating strong market momentum.
- c) The 7-day moving average confirmed a steady growth trend.
- d) Trading volume spikes indicate periods of increased investor activity.
- e) The volatility observed in the stock prices remained moderate, indicating relatively stable market behaviour.

Conclusion

This project demonstrates how SQL and Tableau can be used together to analyse financial datasets and visualize stock market trends.

Using SQL, important metrics such as daily return, average price, and trading volume were calculated. Tableau was then used to visualize these results through charts and dashboards.

Overall, the analysis provides valuable insights into stock performance and helps in understanding market trends.

References

Dataset source: Stock market historical dataset.